Musical Note Classification

Robert Zando, Evelyn Yaskin, Jason Ingersoll

zando.r@northeastern.edu, yaskin.e@northeastern.edu, ingersoll.j@northeastern.edu

1 Abstract

This report covers aspects of both music theory and machine learning. In our study, we perform a multi-class classification of individual notes, determining which of 12 possible note classes the sound files under evaluation fall into. To determine the best approach to this problem, we compared a Convolutional Neural Network, a Feed-Forward Neural Network, and a Support Vector Machine approach in terms of overall model accuracy, as well as per-class F1-scores. We found that the highest accuracy resulted from the neural network approaches, demonstrating a superior capability in handling non-linear feature data. This report is intended to outline the approaches we took to each model, the results from our training/test splits, and discuss the implications of their overall differences in performance.

Keywords: Machine Learning, Classification, Musical Notes

2 Introduction

Our motivation for this project was to apply the principles of various multiclass classification techniques to a problem where there existed a wide variety of potential output classes. In addition, we wanted the problem to be one that offered the possibility of a significant potential pool of data for training. Drawing on our interest in music, the objective we decided on was to find and implement a classification technique that could identify, from an audio file, individual musical notes. This posed a fascinating challenge due to the non-linear nature of sound wave features, as well as determining how to effectively represent this information for machine learning models.

3 Data

There are 12 possible pitch classes that may be clearly represented by individual sound waves, thanks to the concept of enharmonic equivalents (ex. G-sharp and A-flat are identical notes) in Western musical theory. This gives us 12 possible output classes for every input vector. The sound files themselves came from the Freesound and NSynth databases, which are online repositories of hundreds of thousands of individual sounds, including notes played by various instruments. [1][2] The dataset we put together was composed of roughly 12,900 individual sound files, each of either the .wav or .mp3 format, held in a folder and referenced by a .csv "manifest" file. For the data to be interpretable by our models, it was necessary to convert it into a format referred to as a "spectrogram," a visual representation of the frequency and duration of each note (see Figure 1). The Python librosa library has a built-in function capable of converting sound files directly into greyscale mel spectrogram images, whose pixel data may then be used as inputs for different classification approaches. The greyscale format was chosen in order to reduce the overall computational expense of attempting to represent a multi-channel dataset as inputs for our classification methods.

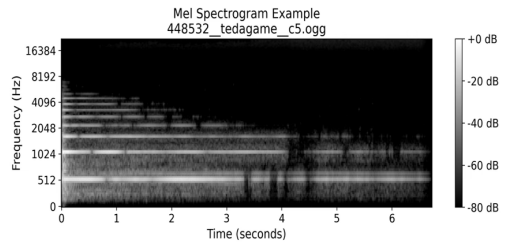


Figure 1: An example mel spectrogram

4 Implementation

The primary focus of our project was on the construction of neural network-based implementations of multi-class classification, which were built using the Pytorch framework. However, in order to more thoroughly explore our options, we also implemented an SVM-based classifier (employing the RBF kernel) using the SVC() function from the sklearn library. In all implementations, we employed the K-Fold cross-validation technique, using a k=5 fold approach. This randomly split the data into a training set representing 80% of the total data, and a test set representing 20% of the data. Each fold was composed of a different combination of the total dataset, and the average performance metrics (accuracy, recall, and F1-score) were gathered for both the training and test sets for the entire fold. Due to the size of the dataset, we wanted to make use of available GPU resources for the neural network training. This was achieved by using the PyTorch framework, which supports the use of a GPU through the NVIDIA CUDA

Class	Predicted	Correct	F1-Score
C	1455	1168	83.00%
C#	910	737	81.70%
D	1223	997	82.10%
D#	923	791	85.10%
E	1042	893	85.70%
F	1137	954	83.70%
F#	901	769	86.50%
G	1233	1075	86.60%
G#	842	754	88.00%
A	1197	1004	83.20%
A#	1041	876	84.40%
B	1050	913	84.30%

Table 1: F1 Scores for all classes in SVM (highest accuracy hyperparameters)

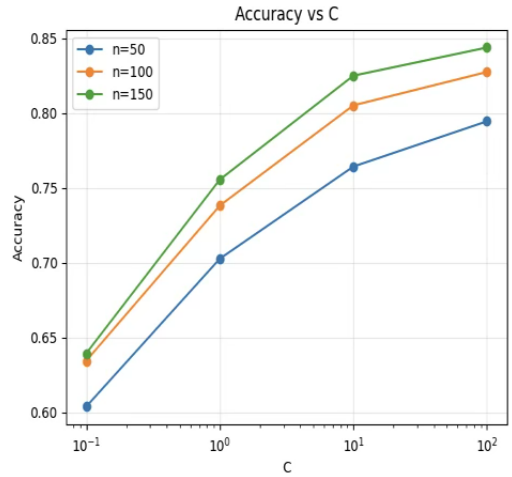
platform, a capability not available to other Python libraries such as Numpy or sklearn.

5 SVM Multiclass Classification

The first method we attempted to apply to this problem was the support-vector machine classification function from the sklearn library. Unlike the following methods, this approach relied on the input data being in the form of a stacked 2D array of all input training data, rather than as batches of 1D vectors. One issue we quickly ran into was the difficulty of applying an SVM approach to a large dataset with so many individual features. A single generated mel spectrogram has an image resolution of 128 pixels X 44 pixels, for a total of 5,632 features per label. Due to the computational cost of SVM-fitting, which is estimated to be between and , simply inputting the raw data from the spectrogram images as a single large Numpy array was not practical. [3] However, as established in prior studies, this problem can be overcome through the application of dimensionality reduction techniques. [4] In the case of our study, we applied principal component analysis (PCA) to reduce the size of the input array, using n=50, 100, and 150 principal components to represent the input datasets (after-the-fact analysis estimates that these components represent 90%, 95%, and 96% of the total variance, respectively).

We trained the model using four different C-regularization values (0.1, 1.0, 10, 100) in order to measure its performance as a function of the hyperparameter, which can be seen in Figure 2. The performance of the highest performing C value (100.0) can be seen in the associated confusion matrix in Figure 3, demonstrating an accuracy of up to roughly 84% on the test dataset using n=150 principal components.

Seen in Figure 3, the dataset shows a mild imbalance, ranging from 6.7% of the total data (G#) to roughly 10.5% of the data (C). Because neither precision nor recall can be considered a more pressing metric for this task, the per-class performance was evaluated using the F-1 Score. In the case of the highest-performing hyperparameter settings, C# had the weakest overall performance and G# had the strongest (Table 1).



t

Figure 2: The accuracy of the SVM classification approach as a function of the regularization parameter (C).

6 Feed-Forward Neural Network (FFN)

This was the approach we had originally proposed at the start of the project. Broadly, our data-preprocessing step "flattens" the converted spectrograms into a 1D vector, where each pixel value is used as a node for the input layer. The hidden layer is a two-layer structure, where we varied the first layer size based on the number of input nodes (0.25x's the number inputs, 0.5x's the number of inputs, 1.0x's the number of input nodes, and 2.0x's the number of input nodes), with the second hidden layer equal to half the size of the first layer.

For the backpropagation step, we employed the Adam optimizer, a commonly employed optimizer for applying the calculated gradients necessary to adjust the weights and biases of the preceding layers during backpropagation. [5] In addition to the hidden-layer size hyperparameter, we also trained the algorithm with three different values for the learning rate hyperparameter (0.0001, 0.001, and 0.01). The hidden layers also included a dropout function, wherein each node output from the first layer had a 20% probability of being zeroed out. The accuracy of the model, as a function of the size of the first hidden layer, can be seen in Figure 4, and demonstrates the overall effectiveness of the FFN approach. While the results for run-throughs where the learning rate was 0.01 were unimpressive, with average training and test accuracy ranging from 11%-14%, training accuracy for the smallest learning rate (0.0001) exceeded 96%, with test accuracy consistently in the 87%-89% range. The lack of sensitivity to hidden layer size strongly implies that the chosen hyperparameters exceeded the critical number of nodes needed to significantly impact model performance. Per-class performance based on F1-score can be seen in Table 2, showing G# once again had the highest performance, while D had

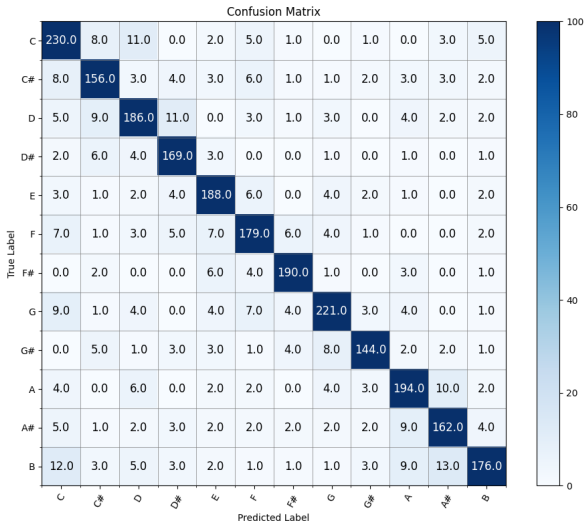


Figure 3: SVM Aggregate Confusion Matrix, n=150, C= 100 parameters

Class	Predicted	Correct	F1
C	1340	1191	88.20%
C#	911	781	86.70%
D	1222	1041	85.70%
D#	949	825	87.50%
E	1077	918	86.60%
F	1148	994	86.80%
F#	921	789	87.70%
G	1230	1105	88.80%
G#	857	797	92.10%
A	1158	1048	88.50%
A#	1054	921	88.10%
B	1085	975	88.60%

Table 2: Per-class performance (FFN)

the lowest.

7 Convolutional Neural Network (CNN)

Convolutional neural networks are a variant of the traditional FFN, which includes one or more "convolutional" layers preceding the typical hidden layers. In this structure, a kernel/filter (i.e., the so-called convolutional layer) generates scaled values from groupings of the original image pixels, and generates a smaller resolution "map" composed of these values representing the original image features. [6] It is widely used in image classification, highly valued due to its ability to capture the correlations which exist between features which are in close-proximity to one another in the image, thereby boosting overall classification performance. In the specific CNN architecture chosen for this project, we used a common 2-layer structure for the convolution component, including a "Maxpooling" step, a downsampling method frequently used in CNNs which reduces the overall resolution of the feature map (thereby reducing overall computational expense). Once the final feature map is generated, it is flattened into a 1D vector and fed into a hidden layer in much the same manner as

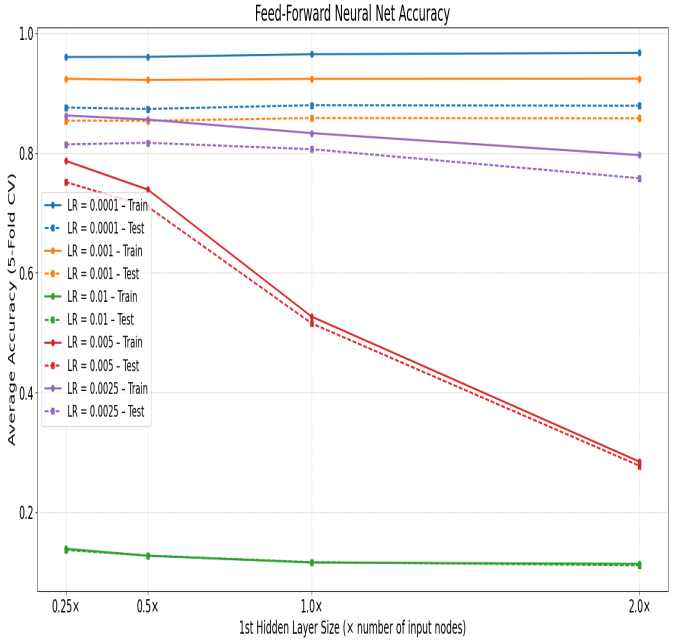


Figure 4: Accuracy of FFN approach as a function of the 1st hidden layer size. Each curve is associated with a different learning rate: 0.0001(Blue), 0.001(Orange), 0.0025(Purple), 0.005(Red), and 0.01(Green)

Class	Predicted	Correct	F1
C	1388	1235	89.90%
C#	909	813	90.10%
D	1179	1075	90.10%
D#	915	855	92.30%
E	1037	964	92.70%
F	1151	1033	90.60%
F#	895	817	92.30%
G	1230	1154	93.10%
G#	854	806	**93.40%**
A	1260	1105	89.30%
A#	1071	951	90.30%
B	1059	1005	91.80%

Table 3: Per-class CNN F1-scores (highest performing hyperparameters)

the traditional FFN. As with FFN, the highest performing (overall accuracy) learning rate was the 0.0001 rate, with an average training accuracy of 98%-99%, and test accuracy of 89%-91%. While the highest learning rate (0.01) also had the worst average accuracy, ranging from 10.5%-38% for training and 10%-38% for test. Moreover, higher learning rates tended to oscillate wildly in terms of overall accuracy, only reaching stability around LR=0.0025, unlike FFN which performed poorly at higher learning rates but performed more consistently.

As seen in Table 3, the highest overall per-class performance of the CNN was comparable to that of the Feed-Forward Neural Network, with G# once again performing the highest and A performing the worst. However, CNN also appears to be much more sensitive to higher learning rates when compared to the FFN (see Figure 6), with wide swings in performance as the size of the hidden layers increase. Additionally, as shown in Figure 8, the conver-

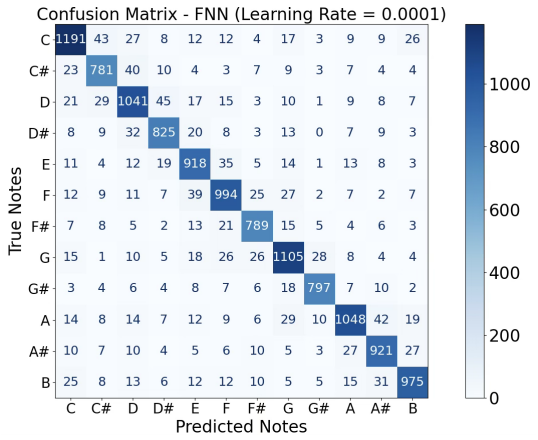


Figure 5: Aggregate FFN Confusion matrix for learning rate = 0.0001 and hidden layer size 2x's that of the input layer

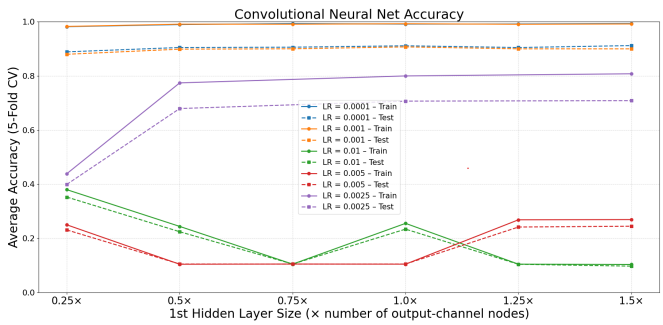


Figure 6: CNN Accuracy as a function of the hidden layer size (based on the number of output channels from the second convolutional layer). Each curve represents a different learning rate: 0.0001(Blue), 0.001(Orange), 0.0025(Purple), 0.005(Red), 0.01(Green)

gence rate of the CNN was much faster in terms of total number of epochs when compared to the FFN for conditions where performance is stable across multiple hidden layer dimensions.

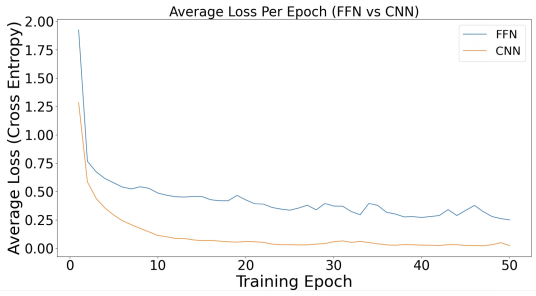


Figure 8: The average loss as a function of the total number of training epochs for FFN and CNN

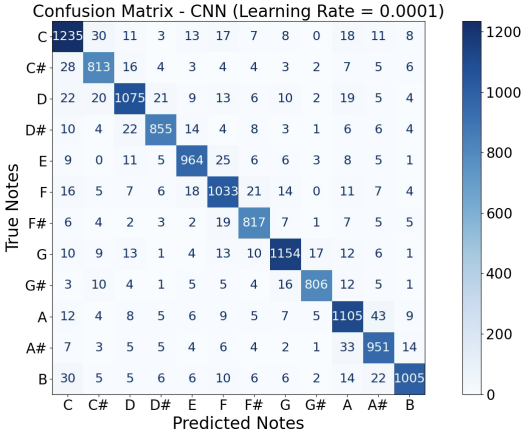


Figure 7: Aggregate Confusion Matrix for CNN

8 Conclusion

Both the FFN and CNN approaches proved to be capable of accurately classifying individual notes saved as sound files. Of the two, CNN converged in fewer epochs when using sufficiently high-performance hyperparameters compared to FFN. However, it also appears to have a greater sensitivity to the learning rate, with its' accuracy oscillating much more than FFN across hidden layer dimensions. While the SVM approach could be used to perform this multiclass classification task, the necessary inclusion of dimensionality reduction, as well as the lower overall maximum accuracy and sensitivity to total represented variance, demonstrates the difficulty in applying this method for large datasets with non-linear features.

With more time, we would have also pursued a multilabel variant of this approach, to identify chords (i.e., collections of notes played simultaneously) from sound files. This would be a fascinating challenge, as sound files incorporating multiple notes played simultaneously would be a substantially more complex dataset to analyze, given the presence of overlapping tonal information creating entirely new, non-linear features and local correlations between pixels.

References

[1] Jesse Engel, Cinjon Resnick, Adam Roberts, Sander Dieleman, Douglas Eck, Karen Simonyan, and Mohammad Norouzi. Neural audio synthesis of musical notes with wavenet autoencoders. *arXiv preprint arXiv:1704.01279*, 2017.

[2] Freesound. Freesound. `https://freesound.org/`, n.d. Accessed: 2026-03-31.

[3] Léon Bottou and Chih-Jen Lin. Support vector machine solvers. In *Large Scale Kernel Machines*, pages 1–28. MIT Press, 2007.

[4] B. Mabrouk, A. B. Hamida, L. Sellami, and N. Jazzar. A comparative study of pca and lda for dimensionality reduction in a 4-way classification framework. *Journal of Applied Material Science & Engineering Research*, 8(1):01–06, 2024.

[5] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[6] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.